Secure Embedded Water Tank Controller

Finley Burns
CYCS 3240 Comprehensive Design
Due November 19, 2026
Prepared November 10, 2026

What I am building

I am building a C17 controller for one simulated water tank, one level sensor, and one inlet valve. The controller will run on an STM32F4 Cortex-M4 in Renode. An operator will use a serial connection to change the low and high thresholds. I will read the test data from files so I can repeat normal operation, invalid input, sensor faults, and command tests.

I kept the two-threshold approach from the preliminary design. The valve opens below the low threshold, closes above the high threshold, and keeps its previous state between the thresholds and at either endpoint. Invalid settings or sensor data close the inlet. Authenticated commands may change the thresholds, but cannot bypass their range checks.

What is complete

The shared C control function, independent host cases, and a sequence runner work. The Windows host tests passed 69 independent cases, 18 sequence steps, 23 parser files, and three command-line error checks. The earlier macOS prototype also passed a Renode test with one fixed 25% reading. The firmware still uses that fixed reading. The changing sensor, timer loop, serial parser, authentication, and tank feedback described here remain implementation work.

How the parts fit together

The C tank model will calculate the water level. A Python test adapter will pass that reading into Renode and read the valve output back. The firmware will check the reading, run the shared controller, and update PD12. The model will use that output to calculate the next volume. Serial commands will enter through USART2, separate from the sensor test interface.

The host programs and firmware will share the same control, input-validation, and command modules. Hardware addresses will stay in the board code. This lets me test most decisions without running an emulator, then check the actual firmware interfaces in Renode.

Scope and tools

The host code will build natively for x86-64 and ARM64 on Windows, macOS, and Linux. The firmware targets Cortex-M4 ARM. I will record each OS and processor separately; a portable build route does not count as a completed test.

I am using C because the course requires C or C++ and I already know C. Host builds use GCC or Clang. Firmware builds use GNU Arm GCC. Renode and Robot Framework provide the emulated board and integration tests [1]. Python handles build commands, emulator coordination, and result collection. Mbed TLS is the planned HMAC-SHA256 library [2]. I still need to select and record its exact release and verify the target build.

There is no network service, dashboard, physical tank, or second device in this project. The serial commands are authenticated but not encrypted. Replay protection is outside the current design. The model will show actuator failures, but the firmware will not claim to detect a stuck valve.

Modules and data

Existing files

controller.h and controller.c define the settings, valve/fault state, validation, and hysteresis decision. **scenario_runner.c** reads independent CSV cases. **sequence_runner.c** keeps state between rows until a reset. The existing firmware files initialize C memory, USART2 output, and PD12. The startup assembly and linker script define the interrupt table and memory layout.

Modules I will add

sensor.c will take a consistent reading and check its level, validity, and age. **serial.c** will hold received bytes, collect bounded lines, and handle timeout or overflow. **commands.c** will parse the command and install both thresholds together. **auth.c** will wrap the selected cryptographic library. **runtime.c** will schedule control and bounded command work. **tank_model.c** will update volume and record empty/overflow events. The Python Renode adapter will coordinate these parts during a scenario.

These names describe the planned module boundaries. They are not claims that the new source files already exist. The November 12 package remains the tested baseline.

Values and ownership

```
tank_config { uint16 low, high }
tank_state  { bool valve_open, fault }
sensor_sample { uint16 level_bp; bool valid; uint32 sample_ms }
runtime { config; state; fault_reason; uint32 next_tick_ms }
serial_state { byte rx[256]; char line[129];
               indices; line_length; first_byte_ms; discard }
model { integer volume_uL; integer inlet_uLps, demand_uLps;
        bool actual_valve_open; integer time_ms }
```

Level is an integer from 0 to 10000 basis points. Settings must satisfy 0 <= low < high <= 10000. The default is (3000,7000). Only the main control loop owns settings and control state. The receive interrupt may add bytes to the queue, but it must not change settings or perform authentication.

The sensor and timer use unsigned 32-bit milliseconds. I will calculate elapsed time with unsigned subtraction, including across wraparound. Intervals must be shorter than 2^31 ms. A timestamp that appears ahead of the current time produces an out-of-range age and is rejected. The host model uses checked 64-bit integers and stops if an arithmetic operation would overflow.

Module interfaces

```
tank_step(state*, config, level_bp, sample_valid) -> void
sensor_read(now_ms) -> sample plus validity/fault reason
serial_poll(now_ms, byte_budget=32) -> zero or one complete line
command_check(line, length, key) -> result plus candidate settings
runtime_tick(now_ms) -> update valve, then process one command
model_step(model*, actual_open, dt_ms) -> next volume and events
```

Errors have explicit results: input error, stale sample, invalid level, bad format, bad tag, bad range, timeout, queue overflow, and missed deadline. The existing public fault flag stays true for any invalid control input. Detailed reasons go to the test log. A command error leaves the current settings alone.

Control and sampling

Startup and periodic work

The board will close PD12 before enabling command input. It will load the default settings, clear the serial state, mark the initial sensor reading invalid, and start a 1 ms SysTick counter. The main loop will run a control step every 100 ms. A timer interrupt will only update the clock and wake the loop. It will not parse input or call the cryptographic library.

```
initialize board with valve closed
load config = {3000, 7000}
clear state, serial buffers, counters
next_tick = now_ms
repeat:
    wait until next_tick is due
    now = read clock
    if now is at least one full period past next_tick:
        close valve; record missed deadline
        next_tick = now + period
        continue
    sample = sensor_read(now)
    tank_step(state, config, sample.level, sample.accepted)
    write valve output immediately
    record control result
    line = serial_poll(now, 32)
    if line is complete:
        result, candidate = command_check(line, key)
        if result is OK:
            config = candidate
        record command result
    next_tick = next_tick + period
```

The control step comes before command processing. An accepted settings change takes effect at the next scheduled control step. Both settings change in the same main-loop operation, so control never sees half of a new pair. If a deadline is missed, the inlet closes and the run records a failure instead of running several delayed steps as if they were on time.

The control decision

```
if settings are invalid, sample is invalid, or level > 10000:
    valve_open = false
    fault = true
else:
    fault = false
    if level < low: valve_open = true
    else if level > high: valve_open = false
    else: keep the previous valve state
```

A sample is accepted only when its validity flag is true, its value is in range, and its age is at most 300 ms. Age 300 is valid and 301 is stale. After a fault, a valid reading clears the fault and follows the same threshold rule. If recovery occurs between the thresholds, the valve stays closed because that was the fault state. Closing the inlet limits further filling, but demand may still empty the tank.

Sensor and board interfaces

Sensor input in Renode

The preliminary design considered the ADC first, with an emulated register as a fallback. For the first full integration, I will use a small shared sensor record in emulated RAM. This gives the test program an exact way to provide the level, validity, and sample time. It tests the firmware control loop, not ADC conversion or electrical behavior. ADC integration can follow if the supported peripheral input can be verified.

The planned linker layout reserves 32 bytes at 0x20000000 for this record and starts ordinary data at 0x20000020. The stack remains at the top of the existing 128 KiB RAM region. The firmware build must verify that data and BSS leave at least 4 KiB for the stack. These addresses are a proposed test interface and are not present in the fixed-reading baseline.

```
0x20000000  uint32 sequence
0x20000004  uint32 level_bp
0x20000008  uint32 valid        // exactly 0 or 1
0x2000000C  uint32 sample_ms
0x20000010  reserved           // remaining bytes unused
```

Before writing a reading, the adapter will stop the emulated machine at a test boundary. It will write an odd sequence number, write all fields, then publish an even sequence number. The firmware will read the sequence, fields, and sequence again. It accepts the snapshot only when both sequence reads match and are even. An inconsistent snapshot is an invalid sample for that cycle. Level and valid fields are checked before any conversion to smaller types.

Board output and serial input

PD12 represents the inlet valve. The firmware will use the GPIO set/reset register to change only that bit. The test adapter will read the output register, so an incorrect UART message cannot hide an incorrect valve output. The existing firmware already initializes USART2 output and PD12 using the STM32F4 registers [3].

The next board version will add USART2 receive on PA3 and enable receive interrupts. The current transmit path uses PA2. Baud rate remains 115200 at the default 16 MHz HSI clock. The startup vector table will name the SysTick and USART2 handlers. I will verify these interrupt and GPIO changes in Renode before claiming that the changing-input firmware works.

Serial queue ownership

The receive interrupt will put one byte into a 256-byte queue and return. The main loop will remove at most 32 bytes per cycle. Producer and consumer positions will use short, protected updates so the consumer cannot read a partially updated queue state. A separate full condition or occupancy count will allow all 256 entries without confusing full and empty.

If the queue fills, the interrupt drops new input and records an overflow flag. The main loop then clears the partial line and discards through a line boundary before accepting another command. Interrupt work stays short. A later test will confirm that incoming traffic cannot delay the 100 ms control schedule.

Command format and authentication

The wire format

```
SET,3500,6500,<64 lowercase hexadecimal tag characters>\n
```

The final byte is LF. Fields use ASCII with no spaces, signs, or leading zeros except the value 0. A line may contain at most 128 bytes before LF. The 129-byte line array includes room for a terminator. HMAC-SHA256 covers exactly SET,3500,6500 in this example, excluding the comma before the tag and the LF. Each hexadecimal pair decodes to one of the 32 tag bytes.

Collecting a line

```
if a partial line exists and now - first_byte_ms > 500:
    clear partial line; record timeout
take at most 32 bytes from the receive queue:
    if discard mode:
        if byte is LF: leave discard mode
        continue
    if byte is LF:
        return this complete line; clear collection state
    if byte is invalid ASCII, NUL, or CR, or length is 128:
        clear partial line; enter discard mode; record error
        continue
    if this is the first byte: save first_byte_ms
    append byte
return no complete line
```

The function returns after one complete line, leaving later bytes queued. At most one authentication attempt runs per cycle. A timeout clears the unfinished command so a later complete line can start again. An oversized or malformed line is discarded through LF so its trailing bytes cannot become a shortened accepted command.

Checking and applying a command

```
require exactly four fields and command name SET
check canonical decimal syntax for low and high
convert each number with checked arithmetic; reject overflow
require exactly 64 lowercase hex characters; decode the tag
verify HMAC over the original command bytes before the tag
if verification fails: return BAD_TAG
if not 0 <= low < high <= 10000: return BAD_RANGE
return OK with both candidate thresholds
```

The main loop installs the candidate only after this function returns OK. I will keep authentication inside a small wrapper around the chosen Mbed TLS interface. The library configuration must include HMAC-SHA256 and its supported tag-verification method. I will pin the release, preserve its license, and run RFC 4231 vectors on both the host and target builds [2,4]. A failed library call rejects the command.

The demonstration key contains bytes 00 through 1f and is public test data. A build intended for another setting would need its own key management. The design assumes the host and emulator are trusted. It provides no encryption or replay prevention, and successful tests cannot establish resistance to every side channel.

Tank model and scenario files

Updating volume

The host model will use integer microliters, integer microliters per second, and integer milliseconds. It will carry the remainder from division by 1000 so a fractional step is not lost on each update. The default 100 L tank begins at 50 L. The inlet supplies 2 L/s when open, and demand starts at 1 L/s. The sensor value is floor(10000 * volume / capacity).

```
net_uLps = actual_open * inlet_uLps - demand_uLps
numerator = checked(net_uLps * dt_ms + saved_remainder)
delta_uL, saved_remainder = signed_divide(numerator, 1000)
raw_volume = checked(volume_uL + delta_uL)
if raw_volume < 0: record EMPTY
if raw_volume > capacity_uL: record OVERFLOW
volume_uL = clamp(raw_volume, 0, capacity_uL)
level_bp = checked(10000 * volume_uL) / capacity_uL
```

The model will record the raw out-of-range value before clamping. The commanded and actual valve states remain separate, so the stuck-open scenario can continue filling even after the firmware commands closure. This does not imply that the firmware senses actual valve position.

Input records

The November 12 files define run settings in **tank_runs.json** and events in **tank_events.csv**. The Python runner will validate the JSON and pass a normalized configuration to the C model. The C model will read event records from a file. Each event names a profile, time in milliseconds, event type, and value. Demand changes happen before the sensor sample at the same timestamp. Events must be sorted by time, use a known type, and fit the selected run.

I will limit scenario durations to 3600000 ms, sample periods to 50, 100, or 500 ms, capacity to 100 L for these tests, and inlet/demand to 0 through 3 L/s. Events must land on a sample boundary. The C reader will reject malformed fields, negative values where disallowed, overflow, extra columns, embedded NUL, and oversized records. These limits cover the test plan and keep arithmetic and file processing bounded.

One integration step

```
load and validate scenario file; reset model and emulator
for each scheduled sample time:
    apply scenario events for this time
    calculate sensor value and any injected fault
    publish the sensor record before the firmware tick
    advance Renode through that control tick
    read PD12 and command/status output
    compare against the independent expected result
    record the full step
    advance the model by one sample interval
```

Time zero is a sampled state, so a 3600-second run at 100 ms produces 36001 samples. The final sample records the state without advancing past the run end. The adapter must distinguish firmware virtual time from host elapsed time. Delayed control output beyond one period is a test failure, even if the volume later returns to its normal range.

Errors testing and completion

What happens after an error

An invalid or old sensor sample, invalid configuration, or missed deadline closes the inlet. Invalid commands leave the existing settings intact. A serial framing error clears or discards the affected line. A model overflow, failed arithmetic check, malformed scenario, or lost emulator connection stops the run with an error and preserves its logs. A missing dependency is reported as not run.

The application will log short reason codes rather than the authentication key. Host process exit status stays 0 for success, 1 for a mismatched result, and 2 for invalid input. Build or tool failures remain separate from expected input rejection. No logging path may block the control loop indefinitely.

How I will check the implementation

The November 12 testing design defines the detailed cases. I will retain C01/C02 and P01 as the regression set, then add S01-S03 for changing and stale samples, A01-A04 for commands and replay limits, T01-T06 for the tank, and L01 for heavy serial input. Expected valve results must come from separately written rules or fixed test data, not from calling the same implementation twice.

I will run the host tests on Windows, macOS, and Linux using the same Python entry point and GCC or Clang. Only the expanded Windows run is verified so far. The Windows GCC installation could not link the sanitizer build because libubsan was missing. The current WSL configuration also failed before Linux started. Those tool limits remain recorded, and neither is a passing test.

Order of the remaining implementation

1. Add the sensor record and timer handlers, then verify changing readings and fault recovery in Renode. Keep the valve closed during startup and confirm the real GPIO state.

2. Add the bounded serial queue and parser. Check every malformed-line and timeout case before connecting authentication.

3. Select and pin Mbed TLS, validate the published HMAC vectors, and add command validation and atomic settings updates. Rerun the control tests after each change.

4. Connect the C tank model and run the normal, demand-change, shortage, stuck-open, long-run, and sampling-rate scenarios. Then add heavy serial traffic and measure virtual-time deadlines.

5. Save the final source and tool versions, run the cross-platform checks that are available, and update the final report and slides with the actual results. The December drafts must remain marked as drafts until these results exist.

References

[1] Renode, Testing with Renode. https://renode.readthedocs.io/en/latest/introduction/testing.html

[2] Mbed TLS project and documentation. https://github.com/Mbed-TLS/mbedtls

[3] STMicroelectronics, RM0090 STM32F4 reference manual. https://www.st.com/resource/en/reference_manual/rm0090-stm32f4xx-reference-manual-stmicroelectronics.pdf

[4] M. Nystrom, RFC 4231, HMAC test vectors. https://www.rfc-editor.org/rfc/rfc4231.html

[5] SEI CERT C Coding Standard, input and memory handling. https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/

I will use CERT C guidance when reviewing conversions, array bounds, strings, and error handling [5]. Project sources are my preliminary design, November 12 testing design, and the saved host results.